

Relazione per Programmazione a Oggetti

HearthCode

Massimiliano Ria
Simone Marsili
Francesco Dama
Lorenzo Mercuriali

Indice

1	Analisi	1
1.1	Descrizione e requisiti	1
1.2	Modello del Dominio	3
2	Design	4
2.1	Architettura	4
2.2	Design Dettagliato	6
2.2.1	Simone Marsili	6
2.2.2	Francesco Dama	13
2.2.3	Lorenzo Mercuriali	13
2.2.4	Massimiliano Ria	14
3	Sviluppo	23
3.1	Testing automatizzato	23
3.2	Note di sviluppo	23
3.2.1	Massimiliano Ria	23
3.2.2	Simone Marsili	24
3.2.3	Francesco Dama	24
3.2.4	Lorenzo Mercuriali	24
4	Commenti Finali	25
4.1	Autovalutazione e lavori futuri	25
4.1.1	Massimiliano Ria	25
4.1.2	Simone Marsili	25
4.1.3	Francesco Dama	25
4.1.4	Lorenzo Mercuriali	25
4.2	Opzionale - Difficoltà incontrate e commenti per i docenti . . .	25
A	Guida Utente	26
B	Esercitazioni di Laboratorio	27

1. Analisi

1.1 Descrizione e requisiti

Il software HearthCode ha l'obiettivo di realizzare una versione semplificata di un gioco di carte strategico a turni ispirato a Hearthstone. L'applicazione appartiene al dominio dei giochi in cui due partecipanti si affrontano utilizzando risorse limitate, carte con caratteristiche specifiche e un insieme di regole che stabilisce quali azioni possono essere eseguite in ogni momento della partita.

A ogni partita partecipano due contendenti: il giocatore umano e un avversario controllato automaticamente dal sistema. Entrambi dispongono di un ammontare di punti vita, di una riserva di mana, di un mazzo da cui pescare e di una mano contenente le carte pescate e giocabili. Lo scontro inizia facendo pescare un determinato numero di carte iniziali a entrambi i giocatori. All'inizio di ogni turno il sistema esegue automaticamente le operazioni obbligatorie di preparazione: aggiorna e rifornisce il mana del giocatore attivo e pesca una carta dal suo mazzo. Se la mano ha raggiunto la capienza massima, la carta pescata non viene aggiunta alla mano e viene invece bruciata. In seguito il giocatore può schierare creature dalla mano al campo di gioco o utilizzare quelle già attive per attaccare il giocatore nemico, danneggiando le sue creature o i suoi punti vita.

Una carta può essere giocata solo se il giocatore possiede una quantità di mana almeno pari al suo costo e se la sua armata non ha già raggiunto il numero massimo di creature consentite. Una creatura appena schierata entra in gioco in stato inattivo e può attaccare solo a partire dal turno successivo del suo proprietario. Dopo avere attaccato, la creatura viene esaurita e non può compiere ulteriori attacchi nello stesso turno.

Per evitare situazioni di stallo, il gioco prevede una penalità nel caso in cui un giocatore debba pescare da un mazzo ormai esaurito. Poiché i mazzi hanno la stessa dimensione e il pescaggio è automatico e obbligatorio per entrambi, l'esaurimento avviene in modo bilanciato nello stesso ciclo di turni. Da quel momento, ogni tentativo di pescaggio a mazzo vuoto danneggia i

punti vita dell'eroe con valori progressivamente crescenti, garantendo che la partita arrivi sempre a una conclusione.

Il sistema è responsabile della gestione dello stato della partita e dell'applicazione coerente delle regole. Di conseguenza, deve impedire mosse non valide, come giocare una carta non presente nella mano, usare una creatura non ancora attiva, attaccare con una creatura già utilizzata o giocare una carta senza mana sufficiente. La partita prosegue attraverso l'alternanza dei turni fino al verificarsi di una condizione di conclusione: quando i punti vita di un giocatore scendono a zero, l'avversario viene dichiarato vincitore.

Requisiti funzionali

- Il sistema deve inizializzare una partita tra il giocatore umano e l'avversario controllato automaticamente, assegnando a entrambi punti vita, mazzo, mano iniziale e risorse di gioco.
- Il sistema deve gestire l'alternanza dei turni, aggiornando correttamente il giocatore attivo, il mana disponibile, il pescaggio obbligatorio e lo stato di attivazione delle creature schierate.
- Il sistema deve permettere al giocatore di giocare carte rispettando le regole definite, in particolare il costo in mana, la presenza della carta nella mano e lo spazio disponibile sul campo.
- Il sistema deve scartare la carta pescata quando la mano del giocatore ha raggiunto la capienza massima.
- Il sistema deve applicare una penalità incrementale ai punti vita del giocatore quando il suo mazzo è esaurito e deve comunque pescare una carta.
- Il sistema deve permettere alle creature attive di attaccare creature avversarie o il giocatore nemico, aggiornando punti vita, stato delle creature ed eventuale rimozione delle creature sconfitte.
- Il sistema deve impedire l'esecuzione di mosse illecite, segnalando l'errore senza compromettere lo stato della partita.
- Il sistema deve gestire autonomamente il turno dell'avversario, scegliendo ed eseguendo azioni valide sulla base dello stato corrente della partita.
- Il sistema deve determinare correttamente la conclusione della partita e il vincitore quando uno dei due giocatori esaurisce i propri punti vita.

Requisiti non funzionali

- Il sistema deve essere portatile: il comportamento dell'applicazione deve rimanere indipendente dall'hardware e dal sistema operativo utilizzati.
- L'applicazione deve rispondere in tempi rapidi alle azioni dell'utente e ridurre i ritardi percepibili, in modo da garantire un'esperienza di gioco fluida.
- Il sistema deve mantenere uno stato di gioco consistente anche in presenza di input non validi, evitando che errori di interazione producano situazioni incoerenti.

1.2 Modello del Dominio

Il sistema pone le basi del modello sulla *Board Game*, concetto che costituisce lo spazio condiviso della partita e coordina lo stato dei due partecipanti. La *Board Game* mantiene il riferimento ai giocatori (*Player*), alle rispettive armate (*Army*) e al turno corrente; inoltre applica le regole principali della partita, come il posizionamento delle carte (*Card*), la gestione degli attacchi, il cambio turno e la verifica delle condizioni di vittoria.

Le carte schierate sul campo sono organizzate in due armate, una per ciascun giocatore. L'armata rappresenta l'insieme delle creature (*Creature*) controllate da un partecipante e distingue quelle attive da quelle dormienti, che devono attendere il turno successivo prima di poter attaccare. Si specifica che una creatura si dice dormiente se appena schierata nell'armata o utilizzata per attaccare.

Ogni giocatore è caratterizzato dai propri punti vita, dalla quantità di mana disponibile, da una mano (*Hand*) e da un mazzo (*Deck*). La mano contiene le carte che il giocatore può scegliere di giocare durante il turno, mentre il mazzo rappresenta l'insieme delle carte non ancora pescate.

Nella versione base del software, la creatura è l'unico tipo di carta realizzata e quindi giocabile. Una creatura è definita da un nome, un costo in mana, un valore di attacco e un numero di punti vita.

Il concetto di intelligenza artificiale infine orchestra le mosse dell'avversario. Tuttavia non è rappresentato tra le entità principali del modello poiché rappresenta più un servizio di dominio e di decisione logica che una sua componente strutturale.

2. Design

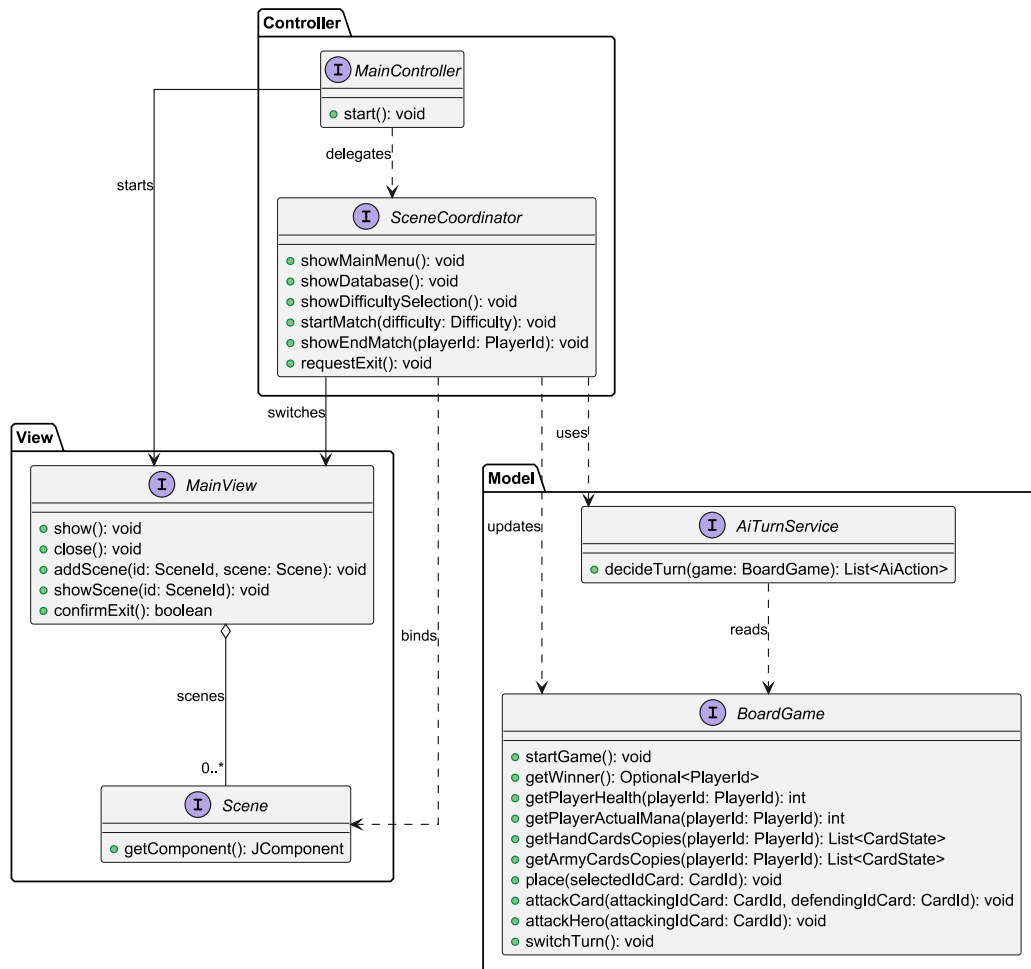
2.1 Architettura

L'architettura dell'applicazione segue il pattern architetturale *Model-View-Controller* (MVC), seppur in una sua variante event-driven con *Model* attivo e aggiornamenti in modalita' *push*. Infatti gli aggiornamenti significativi dello stato della partita vengono comunicati alla scena interessata da parte del *Model* stesso, mentre per il flusso opposto gli input dell'utente vengono intercettati dalla *View* e interpretati dal *Controller*, secondo MVC standard.

L'entry-point principale del *Model* e' rappresentato dall'interfaccia **BoardGame**, che incapsula lo stato e la logica principale della partita. Data la sua composizione ha il compito di applicare le regole del dominio ed eseguire le conseguenti operazioni sulle altre classi. L'interfaccia espone metodi adeguati alla modifica del suo stato e alla sua interrogazione, permettendo al *Controller* di interagire con esso quando uno dei giocatori desidera applicare una determinata mossa. È presente tuttavia un secondo punto di accesso, rappresentato dall'interfaccia **AiTurnService**, che gestisce il servizio di collegamento tra il *Controller* e l'algoritmo decisionale dell'IA.

L'interfaccia **MainView** funge da entry-point della *View* e rappresenta la finestra principale dell'applicazione. Essa tiene traccia delle varie scene dell'applicazione ed è in grado di navigare tra loro grazie a un gestore del layout, mentre il criterio di navigazione è affidato al *Controller*. Per scena si intende una specifica schermata del sistema identificata dall'interfaccia **Scene**. Ciascuna delle scene registrate espone un'interfaccia provvista di metodi specifici che permettono al *Controller* di registrare a un determinato componente grafico una determinata azione. In questo modo la *View* si occupa della rappresentazione grafica, della selezione degli elementi e della raccolta degli eventi generati dall'interfaccia, ma non applica direttamente le regole di gioco. Inoltre **MatchScene** può essere considerato come un punto d'accesso secondario della *View*, in quanto è presente un flusso di comunicazione diretta tra essa e il *Controller* dedicato allo stato della partita.

Il *Controller* è rappresentato principalmente dalle interfacce **MainController** e **SceneCoordinator**. Il controller coordina il flusso applicativo generale, crea le schermate, registra le scene nella vista principale e gestisce la navigazione tra di esse, a richiesta delle azioni utente catturate dalla View. Per evitare un'eccessiva concentrazione di responsabilità, si suddivide le responsabilità in controller specializzati per le singole schermate. Questi collegano le callback esposte dalle View alla logica applicativa corrispondente.



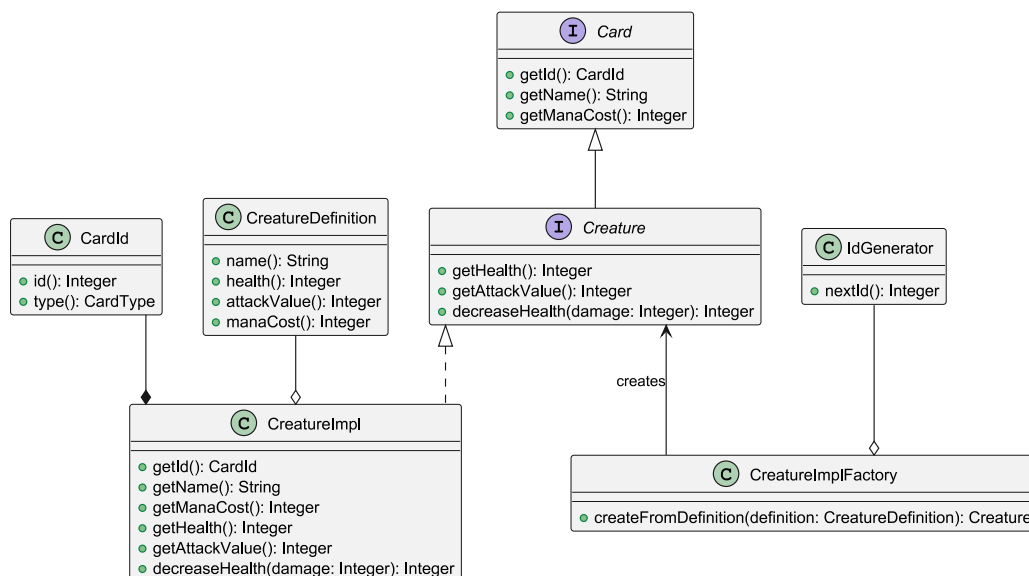
2.2 Design Dettagliato

2.2.1 Simone Marsili

Gestione delle carte di gioco

Problema: Le carte nel gioco possono essere di varie tipologie, come **Creatures** e **Spells** (requisito opzionale del progetto), e il sistema deve essere estendibile a nuove aggiunte e tipi di carte. Inoltre le carte devono contenere un apposito identificatore, il quale permette le operazioni sul modello durante la partita e impedisce la duplicazione di dati tra istanze. Tuttavia è necessario separare l'identità runtime delle carte dalla loro definizione statica. Si è cercato inoltre un modo per separare efficacemente dichiarazioni e implementazioni delle creature.

Soluzione: L'interfaccia **Creature** rappresenta una tipologia di carte schierabili, e l'implementazione **CreatureImpl** si compone di un **CardId**, il quale informa sulla tipologia di carta e sulla sua numerazione univoca nel game, e di una **CreatureDefinition**, che rappresenta invece l'aspetto statico della carta (le sue statistiche base). L'implementazione delle creature è stata realizzata tramite una *Simple Factory*, che rende il codice facilmente estendibile e manutenibile. La factory utilizza un **IdGenerator**, che non istanzia autonomamente, ma che riceve tramite costruttore, riducendo l'accoppiamento e migliorando testabilità ed estendibilità.



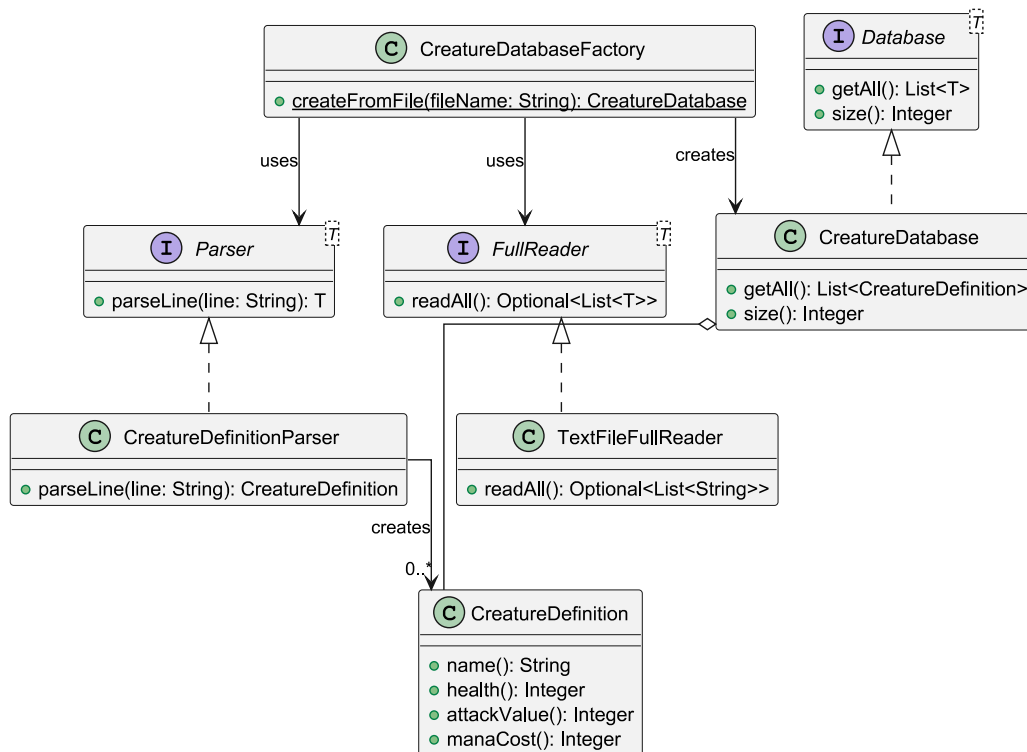
Gestione del database di carte

Problema: Il gioco prevede la presenza di un database contenente tutte le carte che possono comparire in un match. Il sistema necessita di una rappresentazione centralizzata e facilmente estendibile delle carte, caricata da una fonte esterna (file), mantenendo separata la logica di parsing dalla logica di dominio. Inoltre, il database deve gestire esclusivamente dati statici delle carte, risultando indipendente dall'identità runtime delle istanze generate durante la partita.

Soluzione: Il database gestisce esclusivamente definizioni statiche delle carte (**CreatureDefinition**), risultando completamente disaccoppiato dal concetto di identità runtime, che viene invece introdotto solo in fase di creazione delle istanze.

Inoltre la creazione di un'istanza del database avviene tramite apposita Factory statica, che utilizza componenti distinti per la lettura del file e per il parsing del contenuto, mantenendo separate le responsabilità di accesso ai dati e interpretazione degli stessi.

Le interfacce **Parser<T>** e **FullReader<T>** rappresentano due strategie intercambiabili (pattern *Strategy*) rispettivamente per l'interpretazione dei dati e per la lettura delle sorgenti, permettendo di estendere facilmente il sistema a nuovi formati o fonti dati.



Gestione del deck

Problema: Il gioco richiede la creazione di un mazzo da cui il giocatore possa pescare progressivamente le carte durante la partita. Tuttavia, il mazzo non deve essere costruito manualmente né contenere direttamente le definizioni statiche presenti nel database: ogni carta inserita nel deck deve essere una nuova istanza runtime, dotata del proprio identificatore.

Un ulteriore problema riguarda la composizione del mazzo. Non tutte le carte dovrebbero avere necessariamente la stessa probabilità di comparire: ad esempio, carte più forti possono essere rese meno frequenti rispetto a carte più deboli, così da ottenere un deck più bilanciato. Il sistema deve quindi permettere di modificare il criterio di selezione delle carte senza intervenire sulla logica interna del mazzo o sulla factory che lo costruisce.

Soluzione: La gestione del mazzo è stata separata dalla sua costruzione. L'interfaccia **Deck** espone soltanto le operazioni necessarie durante la partita, cioè la pesca di una carta tramite `draw()` e la consultazione del numero di carte rimanenti tramite `getRemaining()`. L'implementazione **DeckImpl** incapsula quindi la lista delle carte effettivamente presenti nel mazzo e si occupa solo della loro gestione runtime.

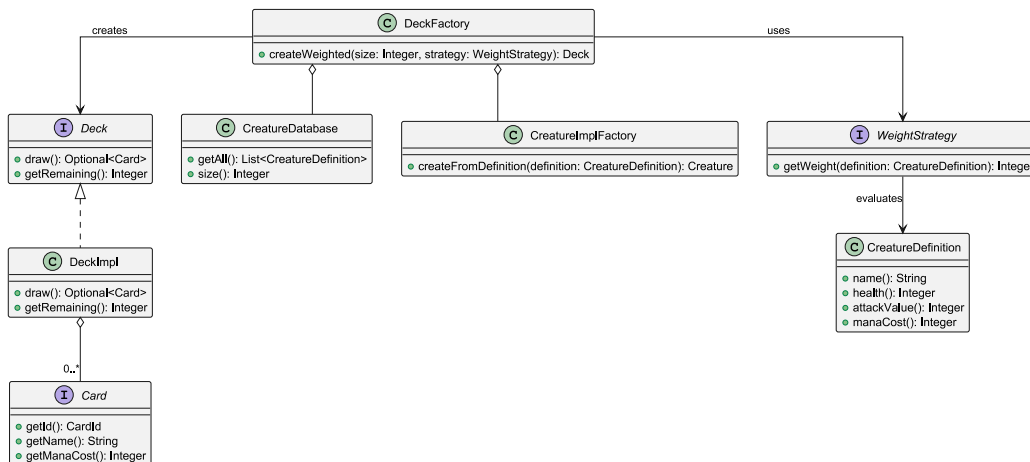
La creazione del mazzo è invece affidata a `DeckFactory`. Questa classe riceve tramite costruttore un `CreatureDatabase`, da cui ottiene le definizioni statiche delle creature, e una `CreatureImplFactory`, usata per trasformare ogni `CreatureDefinition` selezionata in una vera istanza runtime di `Card`. In questo modo il database resta separato dall'identità delle carte usate nella partita, che viene introdotta solo al momento della generazione del deck.

La classe `DeckFactory` realizza sostanzialmente una *Simple Factory*, poiché centralizza la logica di creazione del mazzo e nasconde al resto del modello i dettagli necessari alla sua costruzione (recupero definizioni dal database, creazione istanze runtime, selezione pesata, mescolamento finale...).

Il metodo `createWeighted(size, strategy)` costruisce un mazzo della dimensione richiesta usando una `WeightStrategy`. In questa versione implementata, la factory calcola il peso associato a ciascuna definizione e usa tale valore per effettuare una selezione casuale non uniforme. In seguito le carte generate vengono mescolate prima della creazione del `DeckImpl`, così da evitare che l'ordine del database influenzi l'ordine di pesca.

Nel caso in cui la dimensione richiesta sia maggiore o uguale al numero di carte presenti nel database, la factory inserisce almeno una copia di ogni creatura disponibile e completa poi il mazzo tramite selezione pesata. Questo evita che, in mazzi abbastanza grandi, alcune creature non compaiano mai solo per effetto della casualità. Quando invece la dimensione richiesta è inferiore al numero di definizioni disponibili, l'intero mazzo viene generato tramite selezione pesata.

La soluzione usa il pattern *Strategy* attraverso l'interfaccia `WeightStrategy`. Tale interfaccia rappresenta l'algoritmo intercambiabile con cui viene calcolata l'importanza di una `CreatureDefinition` durante la costruzione del mazzo. `DeckFactory` usa la strategia senza conoscere il criterio concreto adottato, rendendo possibile introdurre in futuro nuove politiche di generazione del deck senza modificare la factory. Inoltre la `WeightStrategy` è una functional interface, il che rende molto facile creare al volo delle nuove strategie.



Gestione dell'aggiornamento della view

Problema: Durante lo svolgimento di una partita, lo stato del modello cambia frequentemente: la partita viene avviata, il turno passa da un giocatore all'altro, vengono pescate o giocate carte, cambiano i punti vita di creature e giocatori, varia il mana disponibile e alcune carte possono essere distrutte o bruciate. Tutti questi cambiamenti devono essere riflessi nella schermata di gioco in modo coerente e tempestivo.

Una prima soluzione possibile sarebbe stata lasciare al controller il compito di aggiornare manualmente la view dopo ogni operazione eseguita sul modello. Questa scelta, però, avrebbe reso il controller responsabile anche dei dettagli di aggiornamento grafico, aumentando il suo accoppiamento sia con il model sia con la view.

Un'altra alternativa sarebbe stata usare una notifica generica, basata su un unico metodo come `update()`. In questo caso, però, la view avrebbe ricevuto solo l'informazione che qualcosa nel modello è cambiato, dovendo poi interrogare il model per ricostruire lo stato aggiornato. Questa variante è stata scartata perché avrebbe richiesto alla view di conoscere più dettagli del modello e avrebbe reso meno esplicito il significato dei singoli aggiornamenti.

Il problema era quindi progettare un meccanismo che permettesse al model di comunicare alla view gli eventi rilevanti della partita, fornendo direttamente le informazioni necessarie all'aggiornamento grafico, senza costringere la view a richiedere ogni volta lo stato aggiornato.

Soluzione: La soluzione adottata applica il pattern *Observer* in una variante di tipo *push model*. Il modello della partita è l'oggetto osservabile, mentre la view della partita è uno degli osservatori registrati. Quando lo stato del model

cambia, il model non invia una notifica generica, ma comunica direttamente quale evento è avvenuto e quali dati aggiornati sono necessari per gestirlo.

L'interfaccia **ObservableGame** definisce il contratto degli oggetti osservabili, esponendo i metodi per registrare e rimuovere osservatori.

L'interfaccia **BoardGame** estende **ObservableGame**, rendendo esplicito che il modello principale della partita può essere osservato. La classe **BoardGameImpl** implementa questo contratto, mantiene gli observer registrati e li notifica opportunamente quando si verifica un cambiamento significativo nello stato della partita.

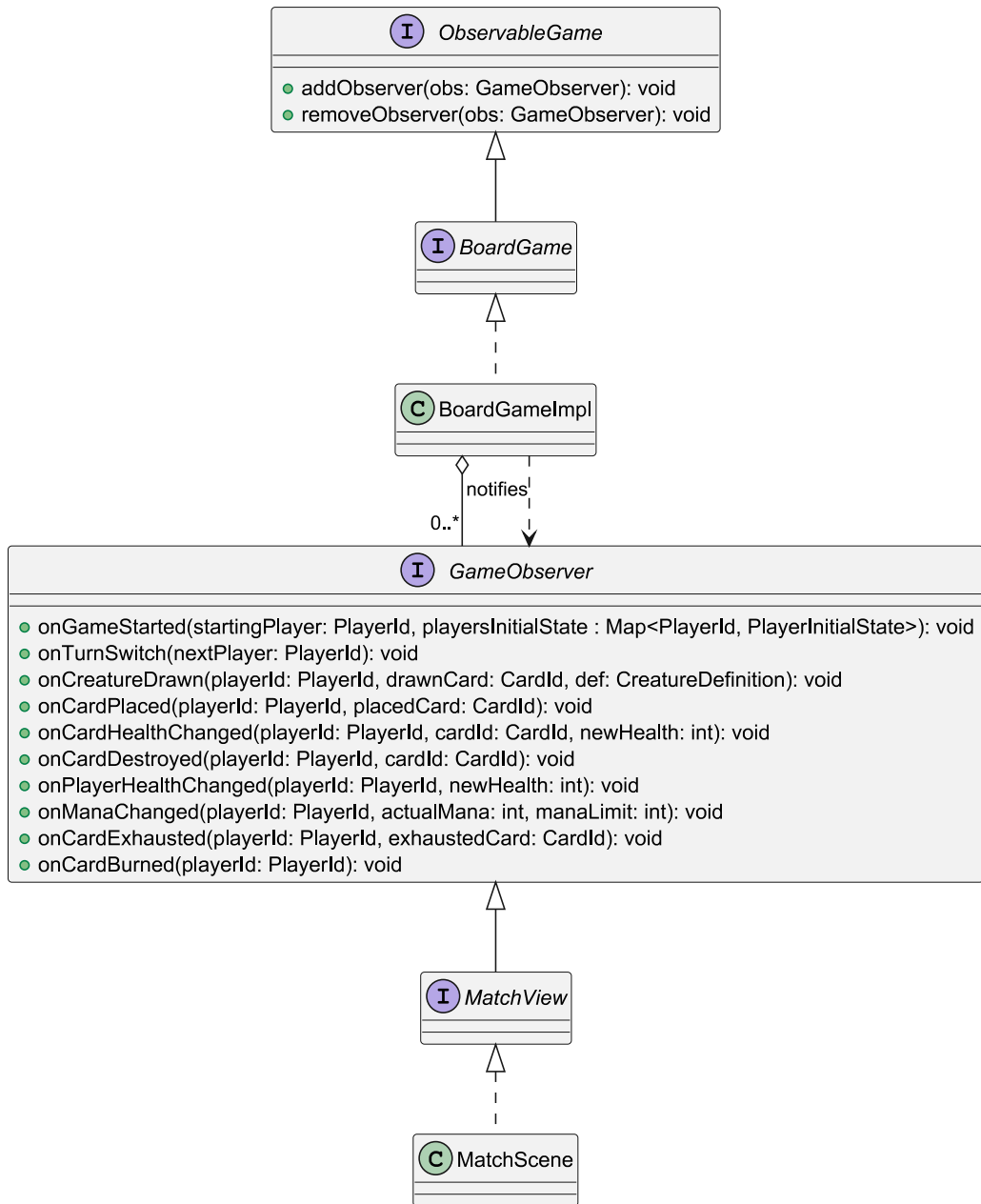
Gli observer sono rappresentati dall'interfaccia **GameObserver**. A differenza di una versione minimale del pattern *Observer*, non è stato usato un unico metodo generico di aggiornamento. L'interfaccia espone invece più metodi specifici, ciascuno associato a un evento rilevante del gioco.

Questa scelta rende la comunicazione tra model e view più esplicita. Il nome del metodo identifica l'evento avvenuto, mentre i parametri trasportano già le informazioni aggiornate.

Sul lato view, **MatchView** estende **GameObserver**. In questo modo, una schermata di partita è anche un osservatore del modello. La classe concreta **MatchScene** implementa **MatchView** e definisce come tradurre ciascun evento ricevuto in un aggiornamento grafico.

Il collegamento tra model e view viene effettuato dal controller, che registra la view come observer del model. Il model, però, non conosce la classe concreta **MatchScene**: conosce solo l'interfaccia **GameObserver**. Questo mantiene basso l'accoppiamento tra le componenti e rispetta la separazione prevista dall'architettura MVC.

La soluzione ha anche un compromesso: l'interfaccia **GameObserver** cresce all'aumentare degli eventi osservabili. Se in futuro venissero introdotti nuovi eventi di gioco, potrebbe essere necessario aggiungere nuovi metodi all'interfaccia e aggiornare le classi che la implementano. Nel contesto del progetto, però, questo costo è accettabile, perché gli eventi rilevanti della partita sono limitati e ben definiti. In cambio, si ottiene una comunicazione più leggibile, più controllata e più aderente al dominio del gioco.



2.2.2 Francesco Dama

Problema 1

Problema:

Soluzione:

Problema 2

Problema:

Soluzione:

Problema 3

Problema:

Soluzione:

Problema 4

Problema:

Soluzione:

2.2.3 Lorenzo Mercuriali

Problema 1

Problema:

Soluzione:

Problema 2

Problema:

Soluzione:

Problema 3

Problema:

Soluzione:

Problema 4

Problema:

Soluzione:

2.2.4 Massimiliano Ria

Adattamento dell'interfaccia a diverse risoluzioni

Problema: Dopo aver ottenuto un'interfaccia grafica funzionante e un'architettura coerente, è emersa la necessità di mantenerne l'aspetto uniforme su risoluzioni e rapporti d'aspetto diversi.

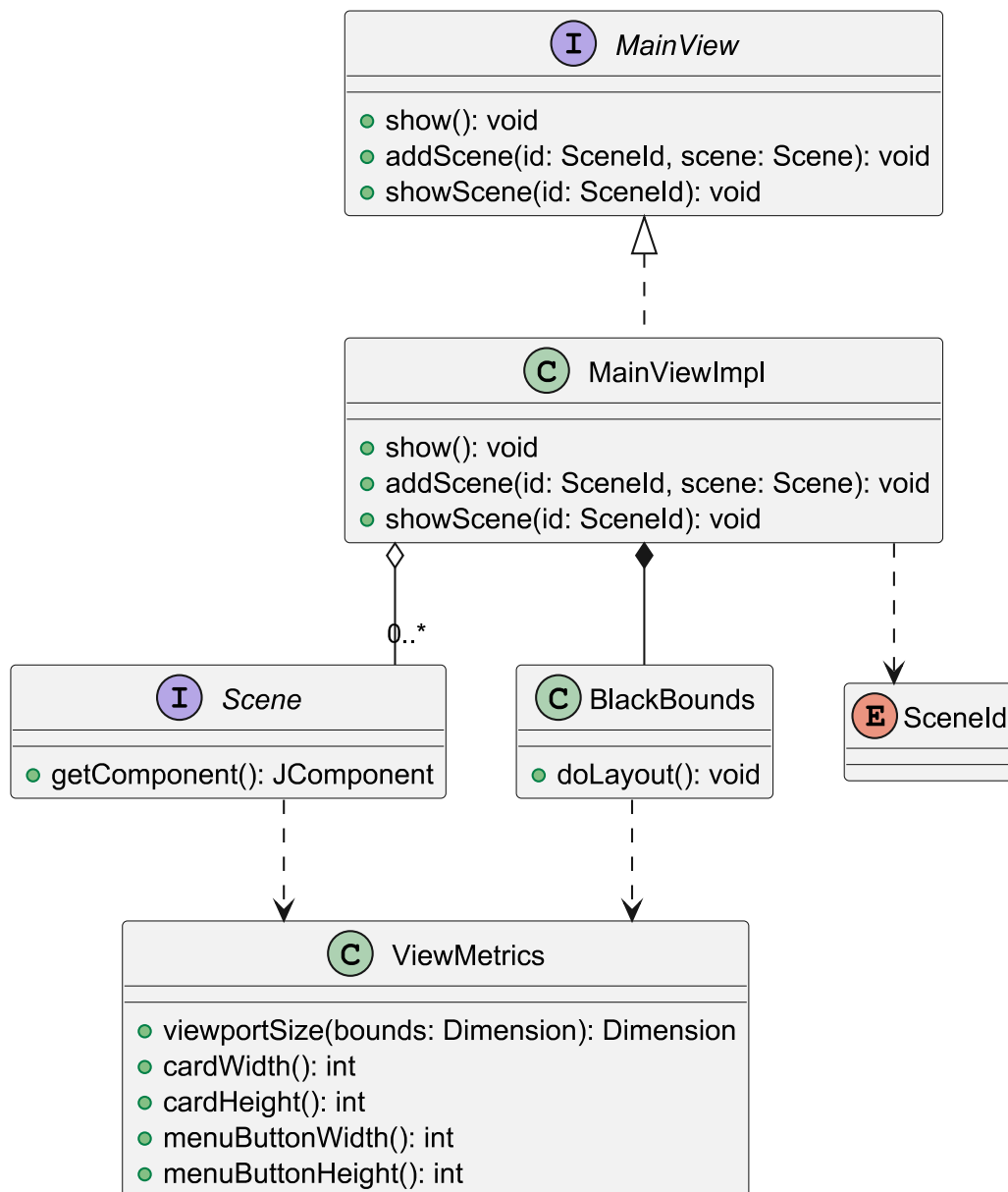
Soluzione: Un primo approccio al problema è stato l'introduzione della classe `ViewMetrics`, usata come punto di accesso unico alle principali dimensioni grafiche dell'applicazione. Nella sua versione iniziale, tali misure venivano calcolate a partire dalla dimensione fisica dello schermo. La classe determina una sola volta le grandezze necessarie e le espone tramite metodi statici dedicati, come ad esempio `cardWidth()` e `cardHeight()`. In questo modo si riduce la duplicazione del codice e ogni scena può richiedere solo la metrica di cui ha bisogno.

I test successivi hanno però evidenziato un limite: scalare ogni componente in base all'intero schermo non garantiva automaticamente una buona visualizzazione. Su schermi con risoluzioni elevate o con un rapporto d'aspetto diverso da quello previsto, alcuni elementi risultavano troppo grandi o disposti in modo poco bilanciato. Inoltre lo sfondo del menu, realizzato nativamente con risoluzione 3840x2160 e rapporto 16:9, rischiava di essere adattato a superfici non coerenti con la sua proporzione originale, producendo tagli dell'immagine o distorsioni impreviste.

Per risolvere il problema è stato introdotto il concetto di *viewport*: l'applicazione continua ad aprirsi a schermo intero, ma la zona effettiva in cui vengono disegnate le scene corrisponde alla più grande area 16:9 contenibile nello schermo dell'utente. Il calcolo è centralizzato nel metodo `ViewMetrics.viewportSize(Dimension)`, che confronta il rapporto tra la larghezza e l'altezza disponibili con il rapporto obiettivo 16:9. Se lo schermo

è più largo del necessario, viene mantenuta l'altezza e ridotta la larghezza; se invece è troppo alto, viene mantenuta la larghezza e ridotta l'altezza.

La classe **BlackBounds** applica concretamente questa scelta: viene usata da **MainViewImpl** come pannello radice della finestra e contiene il pannello delle scene gestito dal **CardLayout**. Nell'override del metodo **doLayout()**, richiamato automaticamente da Swing quando la finestra viene mostrata per la prima volta o quando cambia dimensione, **BlackBounds** calcola la viewport a partire dalla dimensione corrente della finestra, centra l'unico componente figlio e gli assegna l'area 16:9 risultante. Tutto lo spazio esterno rimane nero, generando bande laterali oppure superiori e inferiori in base alla situazione.



Multi-Risoluzione per schermi Hi-DPI

Problema: Nelle ultime fasi di sviluppo è emerso un bug legato alla qualità delle immagini delle carte. Il problema non si presentava in modo uniforme: su alcuni schermi le texture risultavano nitide, mentre su altri apparivano sgranate o sfocate, senza che inizialmente fosse riconoscibile un legame evidente con la risoluzione nominale del monitor.

L'analisi successiva ha ricondotto il comportamento agli schermi Hi-DPI. Su questi dispositivi il sistema operativo può applicare uno *zoom* logico: l'interfaccia continua a ragionare in pixel logici, ma ogni pixel logico viene poi rappresentato tramite più pixel fisici. Questa scelta evita che finestre e componenti diventino troppo piccoli su pannelli molto densi, ma introduce una differenza tra la dimensione richiesta da Swing e la dimensione realmente usata dal dispositivo di output. Nel caso specifico, le classi della View ottenevano le immagini tramite `ImageLoader`, che le restituiva già scalate alle dimensioni calcolate da `ViewMetrics`. Quando però Swing disegnava quelle icone su uno schermo con fattore di scala maggiore di 1, l'immagine già ridotta veniva adattata a una superficie fisica più grande, producendo l'effetto di sgranatura osservato sulle carte.

Soluzione: L'idea adottata è stata quella di non trattare più una richiesta di immagine scalata come una singola bitmap, ma come un'immagine multi-risoluzione. Per ogni risorsa grafica richiesta a una certa dimensione logica viene preparata una variante coerente con quella dimensione e, quando necessario, una seconda variante coerente con la dimensione fisica del dispositivo. In questo modo le classi della View continuano a chiedere immagini in termini di pixel logici, ma il sottosistema grafico di Java può selezionare la variante più adatta al momento del disegno.

Il punto centrale del fix è `AsyncCachedImageRepository`, che implementa `ImageRepository` ed è usata da `ImageLoaderProxy` dietro alla facciata statica `ImageLoader`. Quando riceve una `ImageLoadRequest` scalata, il repository calcola prima la dimensione fisica equivalente tramite il metodo `deviceSize(int, int)`. Questo metodo legge il `defaultTransform` della configurazione grafica principale, ottenuta da `GraphicsEnvironment`, e moltiplica larghezza e altezza logiche per i fattori di scala dello schermo. Nei contesti senza ambiente grafico viene invece usato un fattore 1.0, così il caricamento resta valido anche in esecuzioni headless.

Una volta note le due dimensioni, `AsyncCachedImageRepository` costruisce l'icona nel metodo `createScaledIcon`. La risorsa originale viene letta con `ImageIO` come `BufferedImage`, così può essere ridimensionata in modo controllato. Se dimensione logica e dimensione fisica coincidono, viene restituita una normale `ImageIcon`. Se invece lo schermo richiede più pixel fisici, viene creata una `BaseMultiResolutionImage` contenente la variante logica e quella fisica, poi incapsulata in una `ImageIcon`. Le chiavi della cache delle immagini scalate includono sia la dimensione logica sia quella fisica, evitando di riutilizzare per errore la stessa bitmap su schermi con fattori di scala diversi.

Il ridimensionamento non usa più direttamente `getScaledInstance`, ma

una procedura esplicita basata su `Graphics2D`. Il metodo `scaleImage` riceve l'immagine sorgente e la dimensione obiettivo, poi la riduce con passi intermedi: finché la distanza dal risultato finale resta elevata, la bitmap viene dimezzata; solo alla fine viene prodotta la dimensione esatta. Ogni passaggio è disegnato da `renderScaledImage` con interpolazione bicubica, qualità di rendering elevata e antialiasing. Questa scelta aumenta leggermente il costo del precaricamento, ma concentra il lavoro in un punto controllato e produce una qualità più stabile.

Le classi che mostrano effettivamente le carte non devono conoscere il dettaglio Hi-DPI. `CardComponent` continua a caricare fronte e retro tramite `ImageLoader`. Le misure restano quelle di `ViewMetrics`; nel metodo `paintComponent` il componente disegna l'immagine ricevuta nello spazio del bottone. Analogamente `CardsPanel` usa lo stesso caricatore per costruire gli `IconPanel` del database. Il catalogo `ImagePreloadCatalog` produce invece le richieste scalate per carte, bottoni e banner, così `ImageLoader` può precaricare le varianti prima della visualizzazione delle scene.

Prima di stabilizzare questa soluzione sono stati valutati anche refactor alternativi. In particolare è stato provato un approccio in cui l'immagine ad alta risoluzione veniva mantenuta più a lungo e ridimensionata nel punto più vicino possibile al disegno finale. La qualità visiva era promettente, ma il compromesso risultava meno adatto all'applicazione: il costo dello scaling tendeva a spostarsi sul primo rendering dei componenti, oppure rendeva il precaricamento più pesante e meno prevedibile nelle schermate con molte carte. La soluzione multi-risoluzione centralizzata nel repository è quindi risultata più equilibrata, perché mantiene l'API della View invariata, conserva il precaricamento asincrono già presente e delega a Java la scelta della variante grafica corretta per lo schermo in uso.

Stato Simulabile per l'IA

Problema: L'approccio scelto per l'implementazione dell'IA in fase di progettazione ha da subito evidenziato un concetto inevitabile: per permettere all'IA di valutare l'applicazione di determinate mosse sullo stato della partita, era necessario che queste venissero effettivamente applicate sul tavolo da gioco, ma modificare direttamente lo stato reale del match avrebbe compromesso la partita in corso.

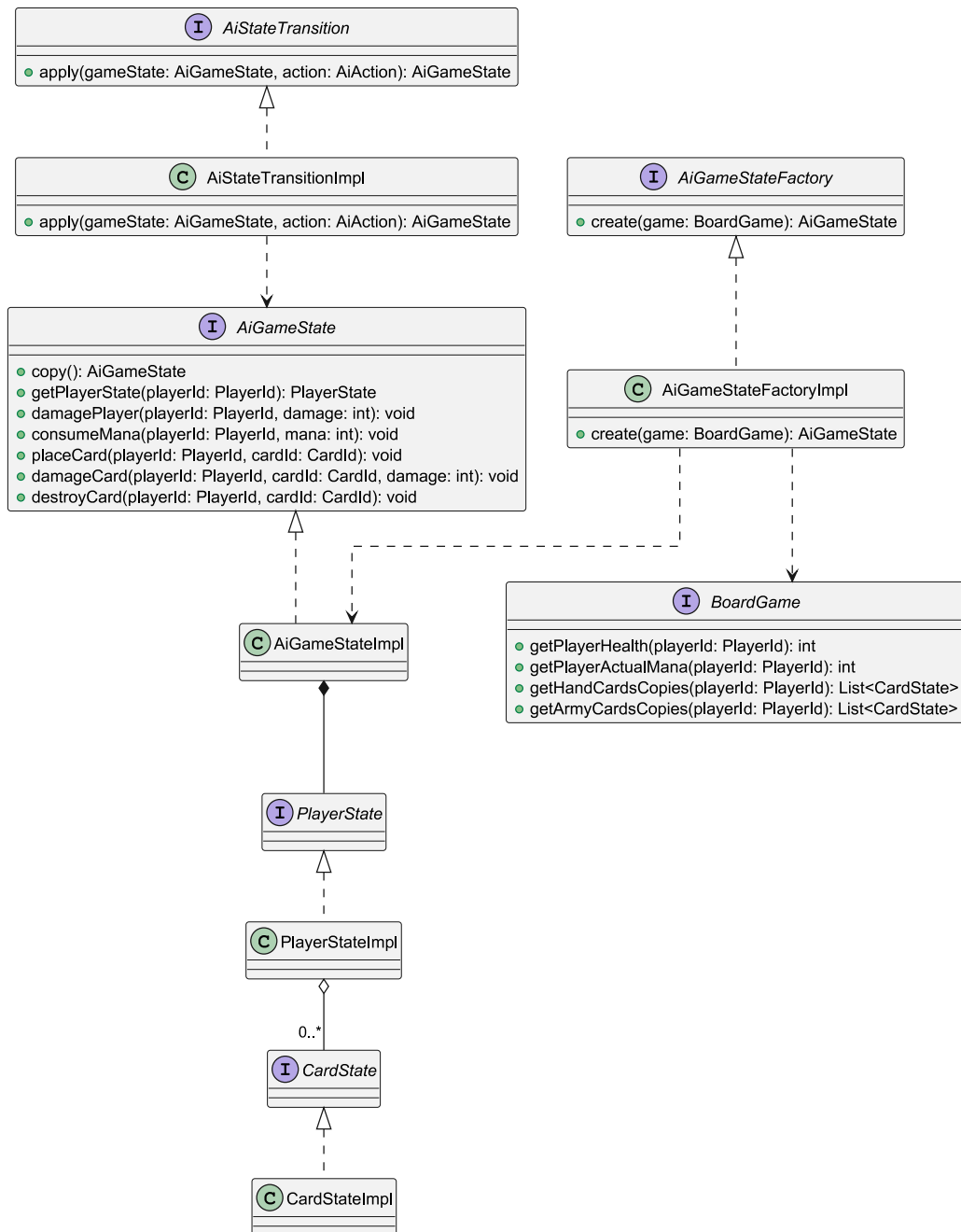
Soluzione: La soluzione più logica è stata dunque introdurre una rappresentazione separata dello stato della partita, ricostruita in un contesto simulato e interamente modificabile. Questo ruolo è affidato all'interfaccia `AiGameState` e alla sua implementazione `AiGameStateImpl`, che espongono le operazioni

minime necessarie alla pianificazione: danneggiare un giocatore, consumare mana, giocare una carta, danneggiare o rimuovere una carta dal campo e produrre una copia dello stato corrente.

La costruzione dello stato simulabile è delegata a **AiGameStateFactory**, implementata da **AiGameStateFactoryImpl**. La factory interroga lo stato reale attraverso **BoardGame** e costruisce lo snapshot copiando i dati dello stato reale e usando copie delle carte. Lo stato del giocatore umano viene creato in maniera dedicata per evitare di copiare dati sensibili che dal punto di vista dell'IA non sono accessibili.

Le informazioni dei singoli giocatori sono isolate dietro **PlayerState** e **PlayerStateImpl**. Questa classe mantiene l'identificativo del giocatore, la sua salute, il mana attuale, l'eventuale mano e l'armata in campo, mentre **CardStateImpl** rappresenta gli stati delle carte copiate.

Infine l'interfaccia **AiStateTransition** completa l'implementazione, rappresentando il concetto di transizione da uno stato simulato a un altro. La classe **AiStateTransitionImpl** infatti crea una copia dello stato ricevuto e poi la modifica in base all'azione da applicare: **PlayCardAction** consuma mana e sposta la carta dalla mano all'armata, **AttackHeroAction** riduce la salute dell'eroe avversario, mentre **AttackCardAction** applica i danni a entrambe le creature e rimuove quelle sconfitte. Gli algoritmi decisionali possono così avere un riferimento a questa classe e utilizzarla per esplorare scenari successivi senza dipendere dalle regole concrete di modifica dello stato.



Politica Decisionale Sostituibile per l'IA

Problema: A un certo punto dell'implementazione dell'IA, è sorta la necessità di integrarla con il ciclo reale del turno. L'algoritmo poteva scegliere una sequenza di mosse su uno stato astratto, ma tali decisioni dovevano poi essere

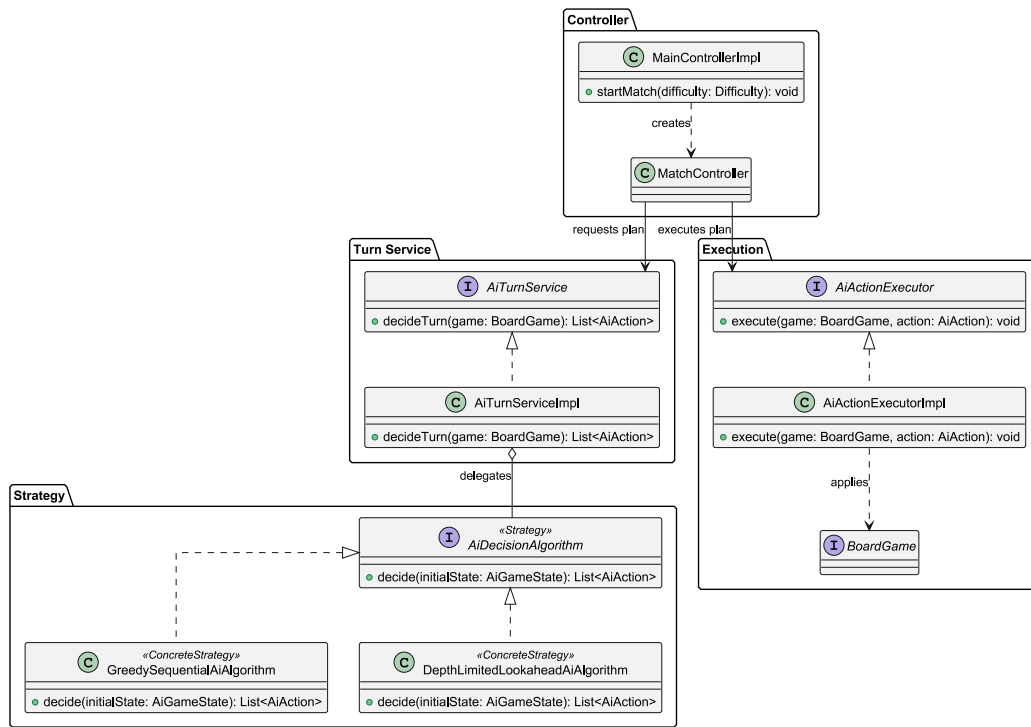
applicate alla partita effettivamente giocata dall'utente. Inoltre la selezione della difficoltà richiedeva di poter sostituire la politica decisionale senza modificare il controller della partita o le regole del modello.

Soluzione: La soluzione adottata separa la scelta delle mosse dalla loro esecuzione reale. Il punto di raccordo principale è rappresentato dall'interfaccia `AiTurnService` e dalla classe `AiTurnServiceImpl`. Il servizio riceve il `BoardGame` corrente, usa `AiGameStateFactory` per costruire lo snapshot simulabile e delega la decisione a un oggetto di tipo `AiDecisionAlgorithm`. Il controller della partita non deve quindi conoscere il funzionamento interno dell'IA: richiede soltanto il piano del turno e ottiene una lista ordinata di `AiAction`.

La sostituibilità della politica decisionale è ottenuta applicando il pattern *Strategy*. L'interfaccia `AiDecisionAlgorithm` definisce il contratto comune degli algoritmi, mentre le implementazioni `GreedySequentialAiAlgorithm` e `DepthLimitedLookaheadAiAlgorithm` rappresentano due strategie alternative. La prima sceglie iterativamente la migliore azione che produce un miglioramento immediato dello stato, la seconda esplora sequenze di azioni fino a una profondità limitata e seleziona quella con la valutazione euristica migliore. Entrambe usano gli stessi componenti di supporto: `AiActionGeneratorImpl` per generare le azioni legali, `AiStateTransitionImpl` per applicarle allo stato simulato e `HeuristicAiStateEvaluator` per assegnare un punteggio agli scenari.

La scelta concreta della strategia avviene in `MainControllerImpl`, dove una mappa associa `Difficulty.NORMAL` a `GreedySequentialAiAlgorithm` e `Difficulty.HARD` a `DepthLimitedLookaheadAiAlgorithm`. Quando l'utente avvia una partita, il controller costruisce `AiTurnServiceImpl` passando l'algoritmo corrispondente alla difficoltà selezionata. In questo modo l'aggiunta di una nuova difficoltà o di un nuovo algoritmo richiede soltanto una nuova implementazione della strategia e la sua registrazione nel punto di creazione.

L'applicazione delle azioni decise dall'IA è invece responsabilità di `AiActionExecutor`, implementata da `AiActionExecutorImpl`. Questa classe traduce le azioni astratte prodotte dagli algoritmi nelle operazioni effettive di `BoardGame`: posizionare una carta, attaccare l'eroe avversario oppure attaccare una creatura. Per farlo invoca rispettivamente `place`, `attackHero` e `attackCard`. Il `MatchController` rimane così il coordinatore del turno, ma non contiene logica decisionale né dettagli di conversione tra azioni simulate e azioni reali.



3. Sviluppo

3.1 Testing automatizzato

Il testing automatizzato e' stato realizzato con JUnit ed eseguito tramite il task Gradle `test`, configurato con `useJUnitPlatform()`. I test sono concentrati sulla parte di modello dell'applicazione. In particolare sono stati testati i componenti che gestiscono giocatore, mano, mazzo, armata, creature e caricamento delle definizioni delle carte. Sono inoltre presenti test per il `BoardGame` e per le componenti dell'intelligenza artificiale, come generazione delle azioni legali, transizioni di stato simulato, valutazione euristica e scelta delle mosse. In questo modo le principali regole della partita e il comportamento automatico dell'avversario possono essere controllati in modo ripetibile a ogni esecuzione della suite.

3.2 Note di sviluppo

3.2.1 Massimiliano Ria

- *Stream + Generics + Lambda Expression + Method Reference*: `AiActionGeneratorImpl.java`, righe 39–48.
- *Stream + Comparator + Lambda Expression + Method Reference*: `GreedySequentialAiAlgoritihm.java`, righe 76–79.
- *Stream + Ricorsione + Comparator + Fallback Lazy + Lambda Expression + Method Reference*: `DepthLimitedLookaheadAiAlgorithm.java`, righe 64–85.

Questa pipeline consente la visita implicita di un albero dove ogni figlio è uno stato del gioco a cui è stata applicata una determinata azione. Grazie a un'implementazione che registra solo determinati nodi durante la visita stessa, non si necessita di classi e librerie avanzate per la costruzione e l'esplorazione di un grafo.

- **Stream + Method Reference + Optional**: `PlayerStateImpl.java`, righe 35–52.
- **Consumer**: `MatchController.java`, righe 100–107.
- **Lambda Expression + Method Reference**: `MatchController.java`, righe 46–51
- **Optional + Java Sound**: `AudioClipLoader.java`, righe 30–47
- **Override** di `paintComponent` per rendering custom con `Graphics2D`: `CardComponent.java`, righe 85–110
- Settaggio di *Rendering Hints*: `CardComponent.java`, righe 166–170
- Utilizzo di classi avanzate quali `AffineTransform`, `BufferedImage`, `BaseMultiResolutionImage` e `GraphicsEnvironment`: `AsyncCachedImageRepository.java`, righe 99–131
- Esecuzione di Azioni IA riconducibile al Pattern *Command*: `AttackCardAction.java`, riga 12 + `AiActionExecutorImpl.java`, righe 19–34.

3.2.2 Simone Marsili

3.2.3 Francesco Dama

3.2.4 Lorenzo Mercuriali

4. Commenti Finali

4.1 Autovalutazione e lavori futuri

4.1.1 Massimiliano Ria

4.1.2 Simone Marsili

4.1.3 Francesco Dama

4.1.4 Lorenzo Mercuriali

4.2 Opzionale - Difficoltà incontrate e commenti per i docenti

Appendice A

Guida Utente

Appendice B

Esercitazioni di Laboratorio